



UNICAMP

Relatório do Trabalho 3: Alinhamento de Imagens Digitais

Relatório desenvolvido a partir da aplicação de técnicas de alinhamento em imagens digitais apresentados nas aulas de MC920 Introdução ao Processamento Digital de Imagem

Prof. Dr. Hélio Pedrini

Letícia Silva Brito
240314

1. Introdução	2
2. Materiais	2
2.1. Requerimentos	2
2.2. Organização das entradas e das saídas	3
2.2.1. Entrada	3
2.2.2. Saída	3
2.3. Execução do Código	3
3. Código	4
3.1. align.py	4
3.2. utils.py	4
3.3. horizontal_projection.py	4
3.4. hough_transform.py	11
4. Eficiência	12
5. Bibliografia	14

1. Introdução

Este trabalho foi feito a fim de estudar como duas técnicas de alinhamento diferentes agem sobre as mesmas imagens e como é possível obter resultados diferentes variando certos valores.

Os códigos que realizaram tais operações, assim como as imagens de entrada e saída, podem ser encontrados no repositório Github: <https://github.com/Asteria101/MC920/tree/main/T-03>.

2. Materiais

Esta seção abordará o que é necessário para a execução e organização do código, de modo a tratar, além do código, a organização das entradas e saídas.

Fragmentou-se o código principal em outros dois arquivos - além do principal - os quais contém as operações requeridas para imagens digitais, abaixo detalhou-se mais sobre cada um deles ao passo que se analisa as técnicas de alinhamento.

2.1. Requerimentos

Este projeto foi desenvolvido na linguagem de programação *Python* na versão 3.12.3, utilizando as seguintes bibliotecas (instaladas em um ambiente virtual, através do comando `pip install ...`), que possibilitaram a criação de algoritmos eficazes para trabalhar com imagens digitais:

Tabela 1: bibliotecas utilizadas e suas respectivas versões

Biblioteca	Versão
OpenCV	4.11.0
NumPy	2.2.4
Matplotlib	3.10.1

A biblioteca *OpenCV* foi utilizada para realizar algumas operações mais específicas com as imagens, assim como as básicas, como ler a imagem em um vetor *NumPy*. Assim, a biblioteca *NumPy* serve para realizar operações de Álgebra Linear e Geometria Analítica com as imagens digitais. Por fim, *Matplotlib* é usada para visualizar e salvar as imagens na máquina.

Demais bibliotecas Python foram usadas como é o caso do *os*, *sys* e *subprocess* de modo a organizar a execução do código.

2.2. Organização das entradas e das saídas

Abaixo detalhou-se como foi feita a ordenação e separação das imagens de entrada e saída de modo a facilitar a chamada e visualização de tais arquivos no diretório.

2.2.1. Entrada

Utilizou-se 7 (sete) arquivos .png que foram usados de entrada e se encontram no diretório *in-images*. As imagens provêm do [diretório](#) de imagens fornecidas pelo Professor Hélio Pedrini. Algumas das imagens apresentam muita similaridade, porém há algumas que permitem analisar a eficiência dos algoritmos mais amplamente.

2.2.2. Saída

As saídas encontram-se no diretório *out-images* e estão divididas conforme os itens do enunciado, de tal maneira que há duas pastas: *horizontal-projection* e *hough-transform*. A primeira pasta contém as saídas para a técnica de projeção horizontal e a segunda para a técnica da transformada de Hough.

As imagens de saída que sofreram a rotação apresentam o prefixo *rot_*; já para as que passaram pela técnica de projeção horizontal há também os histogramas, que apresentam o prefixo *hist_*.

2.3. Execução do Código

A execução do código pode ser feita de duas maneiras: a primeira resguarda-se na execução de um processamento por vez, enquanto a segunda realiza todos os processamentos seguidos.

Isto é para executar manualmente o processamento de uma imagem basta rodar a seguinte instrução no terminal, dentro do diretório **T-03/**:

```
$ python3 align.py <input-image> <mode>
```

Onde *align.py* é o arquivo que será executado, *<input-image>* o nome do arquivo a ser processado com a extensão *.png*, o qual deve estar no diretório *in-images*, e *<mode>* o modo de alinhamento, ou seja, *hp* para Projeção Horizontal e *ht* para Transformada de Hough.

Já para a execução em série, que está estruturada por *execute.py*, deve-se colocar a seguinte instrução no terminal:

```
$ python3 execute.py align.py <input-folder> <mode>
```

Este comando será responsável por receber todos os arquivos do diretório *<input-folder>*, executá-los com base no *<mode>* e guardar as saídas na pasta *out-images*.

3. Código

A boa estruturação de um programa gera uma maior eficiência e facilidade de manutenção, assim, optou-se por dividir o código em alguns arquivos. Abaixo, há a explanação de cada um, contendo a análise do item proposto.

3.1. align.py

Realiza a chamada das funções com base nos modos de alinhamento. Isto é, recebe uma imagem, trata-a para que esteja binarizada utilizando a técnica de Otsu.

3.2. utils.py

Este arquivo contém as importações de bibliotecas necessárias para o funcionamento dos códigos, assim como cinco funções gerais:

- *show(img : np.ndarray);*
- *save(img : np.ndarray, filepath : str);*
- *otsuThreshold(img: np.ndarray);*
- *rotate(img: np.ndarray, angle: float);*
- *getHistogram(horizontal_projection: np.array, height : int, width : int).*

A primeira é responsável por mostrar a imagem na tela, a segunda por salvar a imagem na máquina, a terceira realiza a binarização da imagem, a quarta realiza a rotação com base num ângulo e a última gera um histograma dos pontos pretos ou do texto da imagem.

3.3. horizontal_projection.py

Este arquivo contém uma função, responsável por retornar o ângulo do texto: *projectHorizontally(src_img: np.ndarray) -> int*.

Para detalhar, esta função implementa a técnica de alinhamento com base na projeção horizontal. De modo que se calcula a matriz de rotação para cada ângulo teste (no intervalo de 0 a 180), gera-se uma nova imagem rotacionada e então consegue-se o perfil desta com *np.sum(rotated, axis = 1)*, A partir do perfil realiza-se o quadrado da diferença entre as células adjacentes do perfil, isto é:

`np.sum((hp_rotated[:-1] - hp_rotated[1:]) ** 2)`

Guarda-se esse resultado em um dicionário, cuja chave é o ângulo teste. E ao concluir a testagem de ângulos, retornou-se o ângulo cujo quadrado da diferença entre as células adjacentes do perfil fosse maior. Para garantir que o ângulo estivesse correto, subtraiu-se 180° dos ângulos que fossem maiores que 90°, de modo a abarcar as imagens com rotação anti-horária.

Assim, realizou-se a rotação com base no ângulo fornecido por *projectHorizontally()* e obteve-se seu histograma. Abaixo há alguns resultados e análises.

Imagem rotacionada no sentido anti-horário

Our last argument is how we want to approximate the contour. We use `cv2.CHAIN_APPROX_SIMPLE` to compress horizontal, vertical, and diagonal segments into their endpoints only. This saves both computation and memory. If we wanted *all* the points along the contour, without compression, we can pass in `cv2.CHAIN_APPROX_NONE`; however, be very sparing when using this function. Retrieving all points along a contour is often unnecessary and is wasteful of resources.



Imagem 1: *neg_28.png* imagem original -28°.

Imagem 2: histograma da Imagem 1.

Our last argument is how we want to approximate the contour. We use `cv2.CHAIN_APPROX_SIMPLE` to compress horizontal, vertical, and diagonal segments into their endpoints only. This saves both computation and memory. If we wanted *all* the points along the contour, without compression, we can pass in `cv2.CHAIN_APPROX_NONE`; however, be very sparing when using this function. Retrieving all points along a contour is often unnecessary and is wasteful of resources.



Imagem 3: *rot_neg_28.png* imagem rotacionada, com ângulo +28°.

Imagem 4: histograma da imagem corretamente rotacionada.

Imagem rotacionada no sentido horário

Our last argument is how we want to approximate the contour. We use `cv2.CHAIN_APPROX_SIMPLE` to compress horizontal, vertical, and diagonal segments into their endpoints only. This saves both computation and memory. If we wanted *all* the points along the contour, without compression, we can pass in `cv2.CHAIN_APPROX_NONE`; however, be very sparing when using this function. Retrieving all points along a contour is often unnecessary and is wasteful of resources.



Imagem 5: *pos_41.png* imagem original, 41°.

Imagem 6: histograma da Imagem 5.

Our last argument is how we want to approximate the contour. We use `cv2.CHAIN_APPROX_SIMPLE` to compress horizontal, vertical, and diagonal segments into their endpoints only. This saves both computation and memory. If we wanted *all* the points along the contour, without compression, we can pass in `cv2.CHAIN_APPROX_NONE`; however, be very sparing when using this function. Retrieving all points along a contour is often unnecessary and is wasteful of resources.



Imagem 7: *rot_pos_41.png* imagem rotacionada -41°.

Imagem 8: histograma da Imagem 7.

Como pode-se analisar, consegue-se chegar em uma imagem final com ângulo 0° mantendo-se boa legibilidade, isto é, a binarização mantém uma boa qualidade da imagem, ainda que se note uma considerável perda da suavidade. Na tentativa de conseguir um resultado mais suave, aplicou-se um filtro Gaussiano que também ocasionou na perda de informação, como é possível notar abaixo.

Our last argument is how we want to approximate the contour. We use `cv2.CHAIN_APPROX_SIMPLE` to compress horizontal, vertical, and diagonal segments into their endpoints only. This saves both computation and memory. If we wanted *all* the points along the contour, without compression, we can pass in `cv2.CHAIN_APPROX_NONE`; however, be very sparing when using this function. Retrieving all points along a contour is often unnecessary and is wasteful of resources.

Our last argument is how we want to approximate the contour. We use `cv2.CHAIN_APPROX_SIMPLE` to compress horizontal, vertical, and diagonal segments into their endpoints only. This saves both computation and memory. If we wanted *all* the points along the contour, without compression, we can pass in `cv2.CHAIN_APPROX_NONE`; however, be very sparing when using this function. Retrieving all points along a contour is often unnecessary and is wasteful of resources.

Imagem 9: Imagem 3 após a aplicação do filtro Gaussiano.

Imagem 10: Imagem 7 após a aplicação do filtro Gaussiano.

Agora, analisam-se imagens com conteúdos diferentes das que se encontram acima como uma partitura, um código de barras e uma página de um livro.

Partitura



Imagem 11: *partitura.png* imagem original rotacionada a 26°.

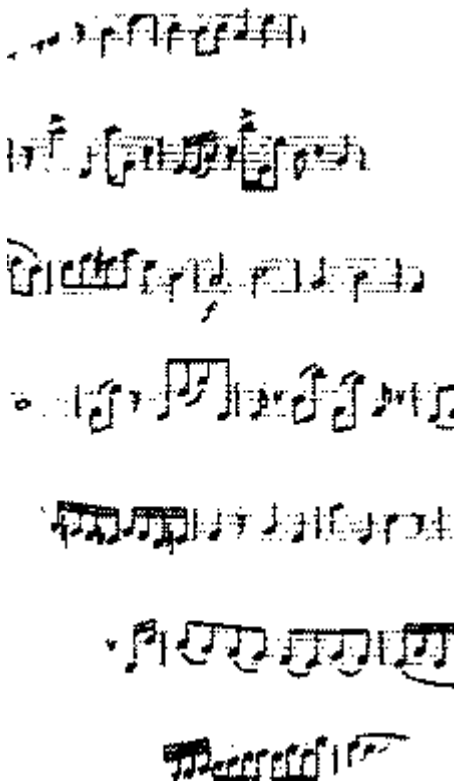


Imagem 12: histograma da Imagem 11.



Imagem 13: *rot_partitura.png* imagem binarizada e rotacionada -26°.

Imagem 14: histograma da Imagem 13.

Nota-se que houve um endireitamento que está de acordo com o esperado, porém a detecção das linhas da partitura não ocorreram como esperado, gerando uma imagem onde se pode definir as notas com alguma precisão, porém sem muita garantia da veracidade apresentada. O mesmo nota-se com o exemplo abaixo.

Código de barras

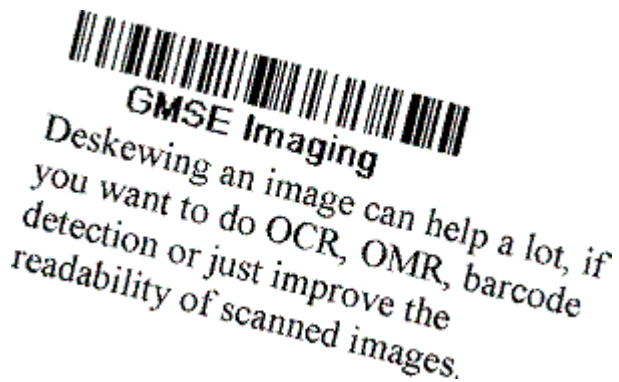


Imagem 15: *sample1.png* imagem original rotacionada a 14°.

Imagem 16: histograma da Imagem 15.



GMSE Imaging

Deskewing an image can help a lot, if you want to do OCR, OMR, barcode detection or just improve the readability of scanned images.



Imagem 17: *rot_sample1.png* imagem rotacionada a -14° .

Imagem 18: histograma da Imagem 17.

Página de livro

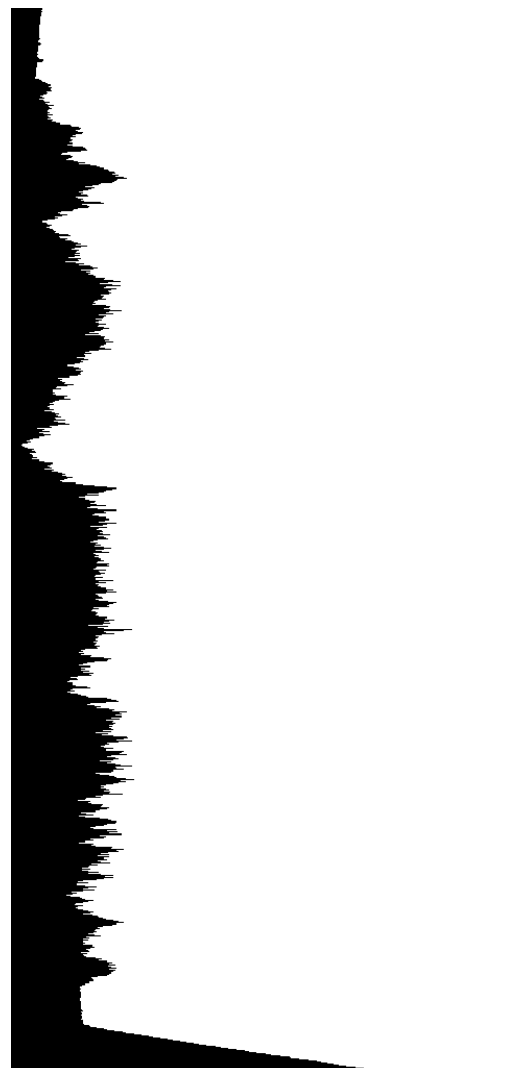
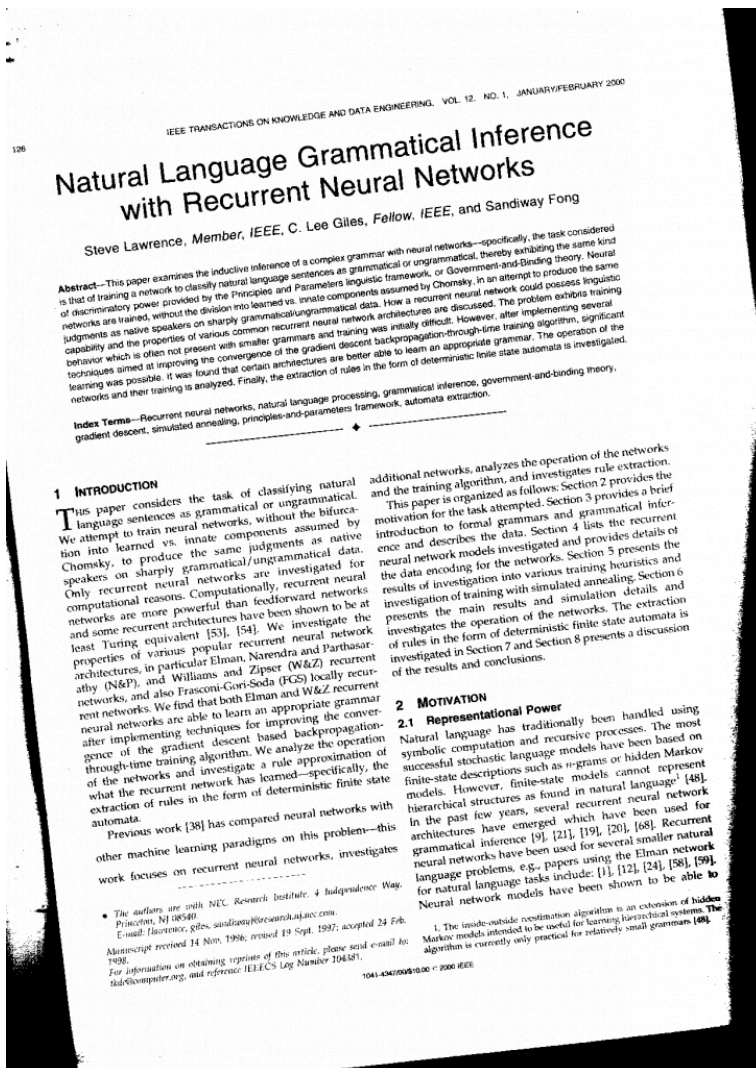


Imagem 19: *sample2.png* imagem original rotacionada a -6° .

Imagem 20: histograma da Imagem 19.

Natural Language Grammatical Inference with Recurrent Neural Networks

Steve Lawrence, *Member, IEEE*, C. Lee Giles, *Fellow, IEEE*, and Sandiway Fong

Abstract—This paper examines the inductive inference of a complex grammar with neural networks—specifically, the task considered is that of training a network to classify natural language sentences as grammatical or ungrammatical, thereby exhibiting the same kind of hierarchical power provided by the Principles and Parameters linguistic framework, or Government-and-binding theory. Neural networks are trained without the device of a separate syntax component assumed by Chomsky, in an attempt to produce the same judgments as native speakers on sharply grammatical/ungrammatical data from a recurrent neural network could produce: linguistic capability and the properties of various common recurrent neural network architectures are discussed. The problem exhibits training behavior which is often not present with similar grammars and training was initially difficult. However, after implementing several techniques aimed at improving the convergence of the gradient descent backpropagation through time training algorithm, significant learning was possible. It was found that certain architectures are better able to learn an appropriate grammar. The operation of these networks and their training is analyzed. Finally, the extraction of rules in the form of deterministic finite state automata is investigated.

Index Terms—Recurrent neural networks, natural language processing, grammatical inference, government-and-binding theory, gradient descent, simulated annealing, principles-and-parameters framework, automata extraction.

1 INTRODUCTION

This paper considers the task of classifying natural language sentences as grammatical or ungrammatical. We attempt to train neural networks without the influence of innate learned syntactic components assumed by Chomsky, to produce the same judgments as native speakers on sharply grammatical/ungrammatical data. Only recurrent neural networks are investigated for computational reasons. Computationally, recurrent neural networks are more powerful than feedforward networks, and some recurrent architectures have been shown to be at least Turing equivalent [33], [34]. We investigate the properties of various popular recurrent neural network architectures, in particular Elman, Narenthra and Parthasarathy (N&P), and Williams and Zipser (W&Z) recurrent networks, and also Frasconi-Covi-Soda (FCGS) locally recurrent networks. We find that both Elman and W&Z recurrent neural networks are able to learn an appropriate grammar after implementing techniques for improving the convergence of the gradient descent based backpropagation-through-time training algorithm. We analyze the operation of the networks and investigate a rule approximation of what the recurrent network has learned—specifically, the extraction of rules in the form of deterministic finite state automata.

Previous work [38] has compared neural networks with other machine learning paradigms on this problem—this work focuses on recurrent neural networks, investigates

additional networks, analyzes the operation of the networks and the training algorithm, and investigates rule extraction.

This paper is organized as follows. Section 2 provides the motivation for the task attempted. Section 3 provides a brief introduction to formal grammars and grammatical inference and describes the data. Section 4 lists the recurrent neural network models investigated and provides details of the data encoding for the networks. Section 5 presents the results of investigation into various training heuristics and investigation of training with simulated annealing. Section 6 presents the final results and simulation details and investigates the operation of the networks. The extraction of rules in the form of deterministic finite state automata is investigated in Section 7 and Section 8 presents a discussion of the results and conclusions.

2 MOTIVATION

2.1 Representational Power

Natural language has traditionally been modeled using symbolic computation and recursive processes. The most successful stochastic language models have been based on finite-state descriptions such as n-grams or hidden Markov models. However, finite-state models cannot represent hierarchical structures as found in natural language [40]. In the past few years, several recurrent neural network architectures have emerged which have been used for grammatical inference [9], [21], [19], [20], [6]. Recurrent neural networks have been used for several smaller natural language problems, e.g., papers using the Elman network for natural language tasks include [1], [12], [24], [58], [59]. Neural network models have been shown to be able to

• The authors are with AT&T Research Institute, 4 Holmdel Place, Holmdel, NJ 08040.

E-mail: {lawrence, giles, sandiway}@research.att.com.

Manuscript received 14 Nov. 1998; revised 16 Sept. 1999; accepted 24 Feb. 1999.

This document is an archival reprint of the article, "Natural Language Grammatical Inference with Recurrent Neural Networks," in IEEE Transactions on Knowledge and Data Engineering, Vol. 12, No. 1, January/February 2000.

Imagem 21: *rot_sample2.png* imagem binarizada e rotacionada a +6°.

Imagem 22: histograma da Imagem 21.

Com uma imagem maior, onde há uma grande quantidade de informações em um espaço pequeno, a perda de pixels é mais notável com a binarização. Para ressaltar isso, atente-se a comparação abaixo.

f classifying natural
tal or ungrammatical.
s, without the bifurca-
ponents assumed by
judgments as native
/ungrammatical data.
are investigated for
onally, recurrent neural
feedforward networks
ave been shown to be at
additional networks, anal-
and the training algorithm
This paper is organized
motivation for the task at
introduction to formal
ence and describes the
neural network models
the data encoding for t
results of investigation
investigation of training
presents the main r

Imagem 23: ampliação da Imagem 19.

or classifying natural
tal or ungrammatical.
s, without the bifurca-
ponents assumed by
judgments as native
/ungrammatical data.
are investigated for
onally, recurrent neural
feedforward networks
ve been shown to be at
|. We investigate the
urrent neural network
latency and Parthasar-
and the training algorithm
This paper is organized
motivation for the task at
introduction to formal
ence and describes the
neural network models
the data encoding for t
results of investigation
investigation of training
presents the main res
investigates the operat
of rules in the form of

Imagem 24: ampliação da Imagem 21.

A considerável perda de informações denotada pela Imagem 24, ressalta que a técnica de binarização funciona para imagens com uma densidade de informações por área menor.

3.4. `hough_transform.py`

Este arquivo possui a função responsável por calcular o ângulo de inclinação da imagem a partir da transformada de Hough, para tal chama-se a função

applyHoughTransform(*src_img* : *np.ndarray*).

Inicia-se o processamento com a criação de um mapa de bordas através da técnica de Sobel, isto é:

```
gx = cv2.Sobel(src_img, ddepth=cv2.CV_32F, dx=1, dy=0, ksize=3)
gy = cv2.Sobel(src_img, ddepth=cv2.CV_32F, dx=0, dy=1, ksize=3)

gx = cv2.convertScaleAbs(gx)
gy = cv2.convertScaleAbs(gy)

# Magnitude of edges
edge_map = cv2.addWeighted(gx, 0.5, gy, 0.5, 0)
```

A partir disso, calcula-se o espaço de Hough de coordenadas (ρ , Θ), primeiramente encontra-se as coordenadas no plano cartesiano e então inicia-se os cálculos para determinar ρ e Θ , sendo possível assim calcular um acumulador que definirá onde tem-se retas. A partir disso, retorna-se o ângulo cujo valor do acumulador é máximo.

Dessa maneira, obtém-se resultados que são muito similares à técnica anterior, como pode ser conferido no diretório *hough-transform*.

4. Eficiência

Como foi demonstrado acima, ocorre uma considerável perda de informações com a binarização da imagem. E isto pode influenciar diretamente na leitura dos textos por ferramentas de OCR, como o Tesseract, que será usado para visualizar a eficiência dos algoritmos. Para armazenar os textos, criou-se a pasta *in-text* que corresponde aos textos de *in-images* e *out-text* que por sua vez corresponde aos textos de *out-images*.

As imagem de entrada *neg_4*, *neg_28*, *pos_24* e *pos_41* apresentam o mesmo texto que é:

*Our last argument is how we want to approximate
the contour. We use cv2.CHAIN_APPROX_SIMPLE to
compress
horizontal, vertical, and diagonal segments into their
end-
points only. This saves both computation and memory. If
we wanted all the points along the contour, without com-
pression, we can pass in cv2.CHAIN_APPROX_NONE;
however,
be very sparing when using this function. Retrieving all
points along a contour is often unnecessary and is
wasteful
of resources.*

Assim, ao rodar o Tesseract sobre elas espera-se obter este texto, no entanto, foi possível identificar texto somente em *neg_4*. Algo que fomenta a hipótese de que há um limite de ângulo de inclinação para que seja possível identificar texto. Algo que pode ser reforçado pelo fato de que não se identificou texto em *partitura* e *sample1*, cujos ângulos são 26° e 14°. Porém, para *sample2* identificou-se texto, ainda que não com muita precisão, como se observa no excerto abaixo.

*4 twrRODUCTION
vs paper conser: the of dassifying cateral aint en a7 OE the networks
ngage sembenecs 28 pramenatical OF sii matical and the Na whee an
Neste cexteaction.
tame to ter neural Se Noithont the bifure 1s paper nergacized 28 FN
Seon 2 provides te
We ante, Feared ve nO vormponets assumed PY metanon oe the task ere
'Setion 3 provides a be
ate me, pigments 2 0 maejacbon 1 formal STATE, eer grammatical HET
ats ae ata ence and ges hl ergata 4
Se spvetigated an POUNCE details
'presents the*

Agora, deve-se analisar como o alinhamento influenciou na leitura dos arquivos pelo Tesseract. Como dito anteriormente, *neg_4*, *neg_28*, *pos_24* e *pos_41* apresentam o mesmo conteúdo só que rotacionado de maneiras diferentes. A nova versão destas

imagens, isto é, a versão paralela ao eixo horizontal, apresentam, uma leitura bem clara do texto, conseguindo chegar em um resultado muito bom.

Com a rotação de *partitura*, não se esperava que a ferramenta de leitura conseguiria identificar as notas, e isto condiz com a realidade, uma vez que, o texto que o Tesseract encontrou é:

ope EFED
af ape
Chet erik LE a
gr Fat gy mg
PRI Sala ad
ACR ADL
in
Cae actal Lea

Anteriormente, *sample1* não havia sido identificado, por conta de seu ângulo de inclinação. Assim, para grau de comparação abaixo encontra-se o texto que se encontra no arquivo *sample1* à esquerda e o texto encontrado após a rotação à direita.



FER EE

GMSE Imaging
 Deskewing an image can help a lot, if
 you want to do OCR, OMR, barcode
 detection or just improve the
 readability of scanned images.

GMSE Imaging
 Deskewing an image can help a lon, i
 you want ta do OCR, OMR, bareade
 detection or just improve the
 readability of scanned images.

Nota-se que houve um problema na identificação de caracteres similares - como **a** e **o**, **c** e **e** - e a notou-se que caracteres muito próximos ao fim da imagem não foram identificados, como é o caso do **f** de *if*. E não se pode desconsiderar a ausência do código de barras.

Por fim, deve-se atentar-se a *sample2* que passou pela rotação. Ao rodar o Tesseract obteve-se:

l faracouenon

ass paper consider the sk of clascitying wtwrad

tangicep? wteters 2 Retnamillew OF ungaaamnakeal
We attempt ur teaus neural petworks. sition! the Mawr
tayn into Joamned vs, ianate compenepis assoined bY
Chomiky, «© predocs the same inuymients 2 ative
apesker on shorply geanepaniaal/mgpanimatic date
Orly reeurrenk seural_astwoehs ate invewigalst tor
romputationad seasons Computational, eocuenent nurs!
errorks ace tonne puwetiid Bap fovdlomard network

*ad seine prcurreny aretubs tutes Bhawe Bae Shown Whe At
 feat Turin oganabeay BSL (Sa), We intestate tae
 properties oF virntis poRMLts eecurrent neural resect,
 reehntectutes, in perliralat Elman, Natenutn aad Pavthaeor
 Ming INES) aml Wiliastas lod Zapser WAZ) ovcurvoat
 hrefweides and also FrascoemetemtSada (FCS) Yecly cect
 rent network We Sid that bath Bhan aunt BED, coset
 Dproral networks atc abe Hera appropriate erat
 ier amplementing Wechanques for imoroning the conver
 genus of the yadiont descent axed backpropagation
 Throughtetne omning,algerthre, We analyze the operation
 of the networks and wrestgan, I rule Bppencnation ot
 'whet the reLurnan! neneick has ranted specials the
 'exbeation of rales wt the: fora af dctesmsimsshe Fete sme
 sutoenate*

Com isto, pode-se concluir que não foi possível identificar informações contundentes que corroboram para a crença de que houve uma melhora após a binarização e rotação da imagem *sample2*. O que pode contribuir para a hipótese da densidade de informações por área.

5. Bibliografia

<https://pyimagesearch.com/2021/05/12/image-gradients-with-opencv-sobel-and-scharr/>